POB AUTOBUILDER · HOW THE STUDIO WORKS

A software studio run by Claude Code agents

PoB Autobuilder is a free, open-source tool that improves Path of Exile builds automatically, with the player's own Path of Building install as the calculator. One person builds it by directing a studio of 30 Claude Code agents. This is how the studio is set up (snapshot: 26 October 2026).

The idea: the main Claude Code session is the **orchestrator**. It does no department work itself. It turns each request into a task file, sends the task through departments modelled on a small game studio, and doesn't call anything done until the gates have produced evidence. **Route, gate, verify.**

Who is in the studio

Orchestrator O — The main session (`.claude/settings.json` → `"agent": "orchestrator"`, max effort). Writes task files, picks the lane, dispatches the six leads, approves their briefs, runs the gates, merges, runs the retro.

Game Interactions — MOST IMPORTANT

interactions-lead O GATE 1 + 4

How skills, items, the passive tree and mechanics really interact in a build. Checks specs before code and results before release; owns the knowledge vault.

- skill-gem-analyst S
- item-analyst S
- tree-ascendancy-analyst S
- mechanics-researcher S
- mechanics-modeler O
- pob-coverage-auditor S
- build-strategist S
- vault-curator S

Features

product-lead S

Turns requests into specs with testable acceptance criteria and a source for every number; keeps the backlog.

- spec-writer S
- user-advocate H – checks specs against the user's past corrections
- data-sources-analyst H – league and price data

UI/UX

uiux-lead S

Makes results understandable and trustworthy in the local web app.

- ux-designer S – flows, wording, empty and error states
- frontend-dev S
- ui-qa H – every value on screen matches the API

Engineering

engineering-lead S

Plans the change, splits it into separate file lists, verifies and commits. Writes no code itself; the coders write tests first, then code.

- pob-engine-dev O – Lua hooks into the calculator
- optimizer-dev S – search, scoring, rules
- integration-dev S – server, caching, data

Code Review

review-lead S GATE 2

CRITICAL and HIGH findings block the work before it reaches QA.

- correctness-reviewer O
- truth-reviewer O – every number traceable to a source
- security-reviewer S

QA

qa-lead S GATE 3

Proves the app works for a player before anything is reported done.

- regression-tester H – golden builds
- api-tester H
- player-sanity-tester S – reads results as an expert player

O Opus: the orchestrator, the interactions lead and the roles where a mistake would hide silently in every number · S Sonnet: most leads and the coders · H Haiku: frequent, mechanical checks. Effort is set per agent as well. Leads only dispatch their own specialists; specialists never delegate.

What the human does

- **Decides only what is theirs:** game facts, trade-offs (damage vs defence vs mana vs comfort) and scope. Questions arrive batched; everything technical the orchestrator decides and records.
- **Types "go"** to run the next task in the queue, and `/clear` after each gate. The task file holds the resume point.
- **Runs blind tests:** holds back a game fact on purpose ("find out yourself"). After the reveal, hit or miss is logged, along with what the research method missed.
- **Gets coached:** a vague request is restated as a task card first: Start / Goal / Done when / Not / Mode (brainstorm or build).

30	6	4	2	40
agents	departments	quality gates	lanes (light / full)	tasks so far

A real exchange, this morning: the user asks to resume a task and says when it counts as done (a benchmark against a build the user corrected by hand, gates 2–4 run). The orchestrator reads the resume point in the task file and checks which agents are still running. The engineering lead hands back: one step committed, one specialist cut off mid-work by the session restart, plus a DISPATCH REQUEST with the exact brief for a fresh specialist to finish it. The orchestrator approves, pre-approves the rest of the chain, settles two technical questions itself and logs everything in the task file. The human isn't asked anything: none of it was a game fact or a scope question.

How a task flows

Light lane (the default, for anything that isn't game logic): build → GATE 2 plus the automated tests → merge → retro.

Full lane (touches game logic: scoring, rules, models, suggestions, the vault, calculator calls): every step below, gates in order.

0 Strategy	1 Spec	GATE 1	3 UI/UX	4 Build	GATE 2	GATE 3	GATE 4	8 Retro
`/plan-ceo-review` for new features and direction changes	product-lead: spec and acceptance criteria	interactions-lead: interaction map and risks	only when the UI changes	`/plan-eng-review`, then engineering, tests first	review-lead: code review	qa-lead: works for a player	interactions-lead: signs off the results	lessons, promotions, token count

also in the light lane. A failed gate sends the work back to the department that caused it, with the findings. After 3 loops the task is marked blocked and the human hears why. Steps that don't depend on each other run in parallel.

What makes it work

One script writes every prompt

`tools/make_agents.py` generates all 30 agent files in `.claude/agents/`: shared project facts, how leads delegate, model and effort per agent. Each rule is tagged with who set it, when, in which task, and which older rule it replaced.

Task files are the memory

`tasks/NNN-slug.md`: front matter (status, departments, the areas it writes) plus a running log. Before every `/clear` the orchestrator writes a resume point, so a fresh session continues where the last one stopped.

Leads ask before they dispatch

A lead ends its turn with "DISPATCH REQUEST" and the exact brief for each specialist. The orchestrator approves or edits it and resumes the lead, which dispatches exactly that.

Every task ends with a retro

Leads report "Lessons:" (trigger → rule → how to check), specialists a "Surprise:" line. Lessons go to `docs/studio/learnings/<dept>.md`, charged to the department that caused the miss, and move into the prompts when they recur.

Isolation by default

One branch and one git worktree per task, one writer per area (engine, vault, golden data). Two tasks run in parallel only when the areas they write don't overlap.

Guardrails in code, not only in prompts

A PreToolUse hook blocks writes to the Path of Building install and the user's builds. A rule that kept being broken (scripts writing into that folder) became a guard in the engine instead of another prompt line.

Evidence or it didn't happen

Nothing is "done" without command output, calculator numbers or a quoted source with its patch version. Every agent checks its work against 8 known failure modes, such as "no change in the calculator doesn't mean no value".

Tests before AI reviews

If a test can check it, a test does. A runner picks the affected test files from the git diff and runs them in parallel (full suite about 70 s instead of 400 s); golden builds pin the numbers.

Token discipline

Most of the cost is context re-read on every call. So: narrow briefs that include the evidence, short-lived agents, replies of at most 15 lines (details go to report files), files read in excerpts, and token usage measured per task.

A knowledge vault for what the engine lacks

An Obsidian vault of rules, models, scenarios and gaps; every note has a machine-readable block and a source. A validator checks it, and game facts stay drafts until the human confirms them.

Want to copy it?

1. Point the main session at an orchestrator prompt whose only jobs are routing, gating and verifying.
2. Generate all agent prompts from one script, with the shared facts in one place; tag each rule with who, when and why.
3. Start with a few departments and real gates. Add a department only when the same kind of miss keeps getting through.
4. Keep work in task files with a log and resume points, and `/clear` often.
5. Give each task its own worktree and have it declare the areas it writes.
6. Turn a rule that keeps breaking into a hook, a test or a guard in code.
7. Run a short retro after every task, and promote a lesson the second time it shows up.

Everything is public: `github.com/koskesh11/pob-autobuilder`. Start with `tools/make_agents.py`, `.claude/agents/`, `tasks/README.md` and `docs/studio/learnings/`.

Snapshot of 26 October 2026. Rules change often; the generator script is always the current version.

Built with Claude Code